SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	UKENTHAR.G	**Reg. No.:**	192524022
Date:	8/10/2026	**Duration:**	60 minutes

Code of Conduct

I UKENTHAR.G (192524022)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ukenthar.G

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format:
     As a <user>, I want <feature>, so that <benefit>.
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks

6. **Estimate Effort Using Story Points**
    - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
    - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

QUESTION 1: IDENTIFY THE PROJECT VISION AND STAKEHOLDERS

1.1 Introduction

The Cloud-Based Online Food Ordering and Delivery System is a software application designed to simplify the process of ordering food from restaurants through the internet. The system allows customers to browse restaurants, view food menus, select food items, place orders, make online payments, and track their deliveries. It provides a common platform for customers, restaurant administrators, delivery partners, and system administrators.

Traditional food ordering methods often require customers to visit restaurants physically or place orders through telephone calls. These methods may result in communication errors, delays, incorrect orders, and difficulties in tracking delivery status. The proposed system addresses these problems by providing an automated and centralized online platform.

Cloud computing technology is used to store and manage application data, making the system accessible through internet-connected devices. The application can be developed incrementally using Agile software development practices.

1.2 Project Vision

The vision of this project is to develop a reliable, secure, scalable, and user-friendly cloud-based food ordering platform that connects customers, restaurants, and delivery partners. The system aims to make food ordering convenient, improve restaurant order management, reduce manual work, and provide timely delivery information.

The application should allow customers to place orders from different locations without visiting restaurants physically. Restaurant administrators should be able to manage menus, prices, availability, and incoming orders. Delivery partners should receive delivery assignments and update order status. System administrators should monitor users and maintain the application.

1.3 Project Objectives

The main objectives of the project are:

1. To develop an easy-to-use online food ordering application.
2. To provide customer registration and secure login facilities.
3. To display restaurant information, food menus, prices, and availability.
4. To enable customers to add food items to a shopping cart and place orders.
5. To integrate an online payment gateway for payment processing.
6. To provide order confirmation and delivery status tracking.
7. To help restaurant administrators manage food items and incoming orders.
8. To maintain customer, restaurant, order, and payment information in a centralized database.
9. To improve the efficiency and accuracy of food ordering and delivery operations.
10. To support future improvements through scalable cloud infrastructure.

1.4 Identification of Stakeholders

Stakeholders are individuals or groups who participate in, use, manage, or are affected by the software system.

1. Customers: Customers are the primary users of the application. They register accounts, log in, browse restaurants, select food items, place orders, make payments, and track deliveries. Their feedback helps improve the application's usability.

2. Restaurant Administrators: They manage restaurant profiles, food menus, item prices, availability, and customer orders. They also update order preparation status.

3. Delivery Partners: They receive delivery assignments, collect prepared food, deliver orders to customers, and update delivery status.

4. System Administrators: They manage user accounts, monitor system operations, configure access permissions, and maintain security and application availability.

5. Payment Gateway Providers: They process online payment requests and return payment confirmation or failure information.

6. Project Manager: The project manager coordinates the development team, plans sprint activities, monitors progress, and resolves project-related issues.

7. Development and Testing Team: Developers design and implement the application, while testers verify functionality, identify defects, and check whether the system satisfies its requirements.

1.5 Expected Benefits

The system reduces manual work, improves order accuracy, provides convenient access to restaurants, and helps customers receive updated order information. Restaurants can organize incoming orders more efficiently, while delivery partners can communicate delivery progress through the application. Cloud infrastructure supports centralized data management and future expansion.

1.6 Conclusion

The project vision establishes the overall purpose and direction of the Cloud-Based Online Food Ordering and Delivery System. Identifying stakeholders helps the development team understand user expectations and business requirements. These details provide a foundation for creating epics, user stories, acceptance criteria, and sprint plans using Agile methodologies.

QUESTION 2: CREATE EPICS AND USER STORIES

2.1 Introduction

In Agile software development, requirements are organized into epics and user stories. An epic represents a large functional requirement that can be divided into smaller and manageable user stories. A user story describes a feature from the perspective of the person who will use it.

User stories help the development team understand what users need and why the feature is important. They also support communication between developers, testers, customers, and product owners.

The Cloud-Based Online Food Ordering and Delivery System contains several major functional areas, including customer management, restaurant management, food ordering, payment processing, delivery tracking, and reporting.

2.2 Epic 1: Customer Registration and Authentication

This epic focuses on creating and managing customer accounts.

User Story 1 – Customer Registration

As a customer, I want to register an account using my personal details so that I can access the food ordering application and place orders.

User Story 2 – Customer Login

As a registered customer, I want to log in securely so that I can access my account, saved information, and previous orders.

User Story 3 – Profile Management

As a customer, I want to update my profile and delivery address so that my personal and delivery information remains accurate.

2.3 Epic 2: Restaurant and Menu Management

This epic allows customers to explore restaurants and view their available food items.

User Story 4 – View Restaurants

As a customer, I want to view available restaurants so that I can choose a restaurant according to my preferences.

User Story 5 – View Food Menu

As a customer, I want to view food names, prices, descriptions, and availability so that I can select suitable food items.

User Story 6 – Manage Menu Items

As a restaurant administrator, I want to add, update, and remove food items so that customers can see the latest menu information.

2.4 Epic 3: Shopping Cart and Order Management

This epic supports the selection of food items and the creation of customer orders.

User Story 7 – Add Items to Cart

As a customer, I want to add food items to a shopping cart so that I can review my selections before placing an order.

User Story 8 – Modify Shopping Cart

As a customer, I want to change item quantities or remove items from my cart so that I can adjust my order before checkout.

User Story 9 – Place Food Order

As a customer, I want to confirm my selected food items and delivery address so that the restaurant can receive and prepare my order.

2.5 Epic 4: Payment and Order Confirmation

This epic manages payments and provides confirmation after an order is submitted.

User Story 10 – Online Payment

As a customer, I want to pay online using a supported payment method so that I can complete my order conveniently.

User Story 11 – Order Confirmation

As a customer, I want to receive an order confirmation with an order number so that I know my order has been recorded.

2.6 Epic 5: Delivery Tracking

This epic helps customers monitor their orders after placement.

User Story 12 – Track Order Status

As a customer, I want to view the latest order status so that I know whether my order is confirmed, being prepared, dispatched, or delivered.

User Story 13 – Update Delivery Status

As a delivery partner, I want to update the delivery status so that customers can receive accurate information about their orders.

2.7 Epic 6: Administration and Reporting

This epic supports platform management and business monitoring.

User Story 14 – Manage Users

As a system administrator, I want to manage user accounts and permissions so that the platform operates securely.

User Story 15 – View Order Reports

As a restaurant administrator, I want to view order and sales reports so that I can understand business performance and plan operations.

2.8 Importance of Epics and User Stories

Epics divide the project into major functional areas, while user stories provide smaller requirements that can be developed and tested within individual sprints. This organization makes it easier to prioritize work, estimate effort, assign responsibilities, and obtain feedback.

The user stories should be reviewed with stakeholders before implementation. Their acceptance criteria must be defined so that the development team can determine when each feature is complete.

2.9 Conclusion

The epics and user stories describe the important functions of the Cloud-Based Online Food Ordering and Delivery System from the perspectives of its users. They provide a clear foundation for acceptance criteria, product backlog prioritization, sprint planning, and effort estimation.

QUESTION 3: DEFINE ACCEPTANCE CRITERIA

3.1 Introduction

Acceptance criteria are specific conditions that a software feature must satisfy before it is accepted as complete. They explain the expected behaviour of a feature and provide a common understanding between the customer, product owner, developer, and tester.

Acceptance criteria should be clear, measurable, testable, and directly related to the corresponding user story. In Agile development, the Given–When–Then format is commonly used to describe expected system behaviour.

- **Given:** Describes the initial condition or situation.
- **When:** Describes the action performed by the user.
- **Then:** Describes the expected result.

The following acceptance criteria are defined for selected user stories in the Cloud-Based Online Food Ordering and Delivery System.

3.2 Acceptance Criteria for User Story 1: Customer Registration

User Story: As a customer, I want to register an account so that I can place food orders online.

Acceptance Criteria 1: Successful Registration

Given that a new customer is on the registration page, when the customer enters all required valid details and submits the form, then the system must validate the information and create the customer account.

Acceptance Criteria 2: Missing Information

Given that the customer leaves a required field empty, when the registration form is submitted, then the system must display a validation message and prevent incomplete registration.

Acceptance Criteria 3: Duplicate Account

Given that an email address is already registered, when the customer attempts to register using the same email address, then the system must display an appropriate message and prevent duplicate registration.

Acceptance Criteria 4: Password Validation

Given that the customer enters a password that does not satisfy the defined password rules, when the registration form is submitted, then the system must request a valid password.

3.3 Acceptance Criteria for User Story 2: Customer Login

User Story: As a registered customer, I want to log in securely so that I can access my account.

1. Given that a customer enters valid login credentials, when the Login button is selected, then the system must authenticate the customer and display the appropriate account page.
2. Given that a customer enters an incorrect password, when login is attempted, then the system must display an error message without revealing sensitive account information.
3. Given that required login fields are empty, when the form is submitted, then the system must request the missing information.
4. The system must restrict account access to authenticated users according to the defined access permissions.

3.4 Acceptance Criteria for User Story 5: View Food Menu

User Story: As a customer, I want to view food items and prices so that I can select suitable food.

1. Given that a customer selects an available restaurant, when the menu page opens, then the system must display the available food items with their names and prices.
2. Given that a food item is unavailable, when the customer views the menu, then the system must clearly indicate its unavailability.
3. Given that the restaurant administrator updates a menu item, when the updated menu is retrieved, then the system must display the latest saved information.
4. The displayed price must match the price stored for the corresponding menu item.

3.5 Acceptance Criteria for User Story 7: Shopping Cart

User Story: As a customer, I want to add food items to a cart so that I can review my order before checkout.

1. Given that a food item is available, when the customer selects Add to Cart, then the system must add the item to the cart.
2. Given that the customer changes an item's quantity, when the cart is updated, then the system must recalculate the subtotal correctly.
3. Given that the customer removes an item, when the cart is refreshed, then the removed item must no longer appear in the cart.
4. The cart must display the selected items, quantities, prices, and calculated subtotal.

3.6 Acceptance Criteria for User Story 9: Place an Order

User Story: As a customer, I want to place a food order so that the restaurant can prepare it.

1. Given that the cart contains valid items and the customer provides a delivery address, when the customer confirms the order, then the system must validate the order details and record the order.
2. Given that the cart is empty, when the customer attempts checkout, then the system must request the customer to add food items.
3. Given that an item becomes unavailable before checkout, when the order is validated, then the system must inform the customer and prevent ordering that unavailable item.
4. The system must generate a unique order identifier for every successfully recorded order.

3.7 Acceptance Criteria for User Story 10: Online Payment

1. Given that the customer proceeds to payment, when a supported payment method is selected and the payment is successful, then the system must record the confirmed payment status.
2. Given that payment fails, when the payment gateway returns a failure result, then the system must display a failure message and must not mark the payment as successful.
3. The system must associate the payment record with the corresponding order.
4. The system must avoid treating repeated payment notifications for the same transaction as separate successful payments.

3.8 Acceptance Criteria for User Story 12: Order Tracking

1. Given that a customer has placed an order, when the order status changes, then the system must display the latest recorded status.
2. The system must support the agreed statuses, such as Confirmed, Preparing, Out for Delivery, and Delivered.
3. Given that an order has been delivered, when the customer views its details, then the system must display the final Delivered status.
4. Customers must only be allowed to view order information associated with their authorized accounts.

3.9 Importance of Acceptance Criteria

Acceptance criteria reduce misunderstandings between developers and customers. They help testers prepare test cases, enable the product owner to evaluate completed work, and establish a clear definition of successful implementation.

3.10 Conclusion

Acceptance criteria provide measurable conditions for evaluating the features of the food ordering system. Using the Given–When–Then format makes expected system behaviour clear and supports reliable testing, sprint reviews, and quality assurance.

QUESTION 4: PRIORITIZE THE PRODUCT BACKLOG

4.1 Introduction

The product backlog is an ordered list of features, improvements, technical work, and other requirements needed to develop a software product. The product owner is responsible for maintaining and prioritizing the backlog based on customer needs, business value, dependencies, risks, and feedback.

In Agile software development, the product backlog is continuously refined as new requirements are identified. High-priority items are generally considered for earlier sprints, while lower-priority features can be implemented later.

For the Cloud-Based Online Food Ordering and Delivery System, backlog prioritization ensures that essential functions are developed before optional improvements.

4.2 Objectives of Backlog Prioritization

The main objectives are:

1. To identify the most important customer requirements.
2. To deliver valuable features as early as possible.
3. To establish a logical development sequence.
4. To consider dependencies between related features.
5. To use available development time efficiently.
6. To reduce risks associated with essential functionality.
7. To ensure that the first sprint provides a useful foundation for later development.

4.3 Prioritization Technique: MoSCoW

The MoSCoW technique divides requirements into four categories.

Must Have: Essential features required for the product's core operation.

Should Have: Important features that improve the system but may not be necessary for the initial working version.

Could Have: Useful features that can be implemented if sufficient time and resources are available.

Won't Have in the Current Release: Features deliberately postponed to a future release.

4.4 Prioritized Product Backlog

Priority	User Story	Requirement	Category
1	US1	Customer registration	Must Have
2	US2	Customer login	Must Have
3	US4	View restaurants	Must Have
4	US5	View food menu	Must Have
5	US7	Add items to shopping cart	Must Have
6	US8	Modify shopping cart	Must Have
7	US9	Place a food order	Must Have
8	US10	Online payment	Must Have
9	US11	Order confirmation	Must Have
10	US6	Restaurant menu management	Should Have
11	US12	Order status tracking	Should Have
12	US13	Delivery status updates	Should Have
13	US3	Customer profile management	Should Have
14	US15	Order and sales reports	Could Have
15	US14	Advanced user administration	Could Have

The table provides a proposed order for planning. The product owner may change priorities after reviewing stakeholder feedback, technical dependencies, and release requirements.

4.5 Justification for Prioritization

Customer registration and login receive high priority because they establish the foundation for account-based services. Restaurant listings and food menus are also essential because customers need to identify restaurants and choose items before ordering.

The shopping cart must be developed before checkout because customers need to review their selected items and quantities. Order placement depends on the cart and the availability of valid menu items. Payment processing and order confirmation are essential for completing the main ordering workflow.

Menu management is important because restaurant administrators need to maintain accurate item information. Delivery tracking and delivery status updates improve transparency after an order is placed.

Sales reports and advanced administration can be scheduled later because they are not required to demonstrate the basic customer ordering workflow. However, essential security and access-control requirements must be addressed from the beginning rather than postponed as optional features.

4.6 Sprint Planning Based on Backlog Priority

For the first sprint, the team can select customer registration, login, restaurant listing, and menu display. These features create the initial customer-facing application.

The second sprint can focus on shopping cart functionality and order placement. The third sprint can include payment integration, order confirmation, and order tracking. Later sprints can enhance restaurant management, delivery operations, reporting, and administrative functions.

The final selection must depend on the team's capacity, technical dependencies, and the acceptance criteria of the selected stories.

4.7 Benefits of Product Backlog Prioritization

Backlog prioritization improves transparency and helps the team understand which features should be developed first. It also reduces the possibility of spending time on less important features before the essential workflow is available.

Regular backlog refinement allows the product owner to respond to customer feedback and changing business requirements without losing sight of the project's overall objectives.

4.8 Conclusion

Product backlog prioritization helps the development team organize requirements according to value, urgency, dependencies, and feasibility. The MoSCoW technique provides a structured approach for distinguishing essential features from optional improvements. A well-prioritized backlog supports realistic sprint planning and incremental delivery of the food ordering system.

QUESTION 5: PLAN THE SPRINT BACKLOG

5.1 Introduction

Sprint planning is an Agile activity in which the Scrum Team decides what work can be completed during the upcoming sprint and how that work will be performed. The sprint backlog contains the selected product backlog items, the sprint goal, and the actionable tasks required to deliver the selected functionality.

A sprint is a fixed-length development period. For this project, a two-week sprint is proposed. The team selects work according to business priority, dependencies, complexity, available capacity, and its Definition of Done.

The first sprint of the Cloud-Based Online Food Ordering and Delivery System focuses on establishing the basic customer-facing functionality.

5.2 Sprint Details

Sprint Number: Sprint 1

Sprint Duration: Two weeks

Sprint Goal: Develop the initial customer-facing application that allows users to register, log in, view available restaurants, and browse food menus.

Selected User Stories:

- US1 – Customer registration.
- US2 – Customer login.
- US4 – View restaurants.
- US5 – View food menu.

These stories are selected because they establish the basic account and browsing functionality required before implementing the shopping cart and checkout process.

5.3 Sprint Backlog and Task Breakdown

User Story 1: Customer Registration

User Story: As a customer, I want to register an account so that I can access the food ordering application.

Development Tasks:

1. Identify the required customer information.
2. Design the registration page.
3. Implement form fields and input validation.
4. Create the customer database table.
5. Implement account creation and duplicate-email validation.
6. Display appropriate success and error messages.
7. Test registration with valid, incomplete, and duplicate information.

Expected Result: Customers can register using valid details, and the system handles invalid input appropriately.

User Story 2: Customer Login

User Story: As a registered customer, I want to log in securely so that I can access my account.

Development Tasks:

1. Design the login interface.
2. Implement credential validation.
3. Connect the login process to the customer database.
4. Configure secure password handling.

5. Display appropriate messages for invalid credentials.
6. Restrict access to authenticated customer functions.
7. Test successful and unsuccessful login attempts.

Expected Result: Registered customers can log in successfully, while invalid login attempts are handled securely.

User Story 3: View Restaurants

User Story: As a customer, I want to view available restaurants so that I can choose where to order food.

Development Tasks:

1. Identify the restaurant details to display.
2. Create the restaurant database table.
3. Design the restaurant listing interface.
4. Retrieve available restaurant records.
5. Display restaurant names and relevant information.
6. Handle empty results and data retrieval errors.
7. Test restaurant listing and display.

Expected Result: Customers can view available restaurants through a clear interface.

User Story 4: View Food Menu

User Story: As a customer, I want to view food items and prices so that I can select suitable food.

Development Tasks:

1. Identify required food menu fields.
2. Create the food menu database table.
3. Design the menu display page.
4. Retrieve food items associated with a restaurant.
5. Display food names, prices, and availability.
6. Handle unavailable items and empty menus.
7. Test menu retrieval and displayed information.

Expected Result: Customers can view the food menu and identify available items and their prices.

5.4 Two-Week Sprint Schedule

Week 1: Planning and Core Development

- Day 1: Sprint planning, requirement clarification, and task allocation.
- Days 2–3: Registration interface, database design, and account creation.
- Days 4–5: Login functionality and initial testing.

Week 2: Restaurant Browsing and Testing

- Days 6–7: Restaurant listing implementation.
- Days 8–9: Food menu display and database integration.
- Day 10: Integration testing, defect correction, sprint review, and retrospective.

This schedule is a proposed plan. The team can adjust task allocation when technical difficulties or unexpected dependencies arise.

5.5 Team Responsibilities

The product owner clarifies requirements and acceptance criteria. The Scrum Master facilitates sprint activities and helps remove obstacles. Developers implement the features and database operations. Testers verify functionality and report defects. Team members collaborate to ensure that the selected user stories meet the Definition of Done.

5.6 Sprint Monitoring

The team conducts a short Daily Scrum to inspect progress toward the sprint goal, identify obstacles, and plan the next day's work. A task board can be used with columns such as To Do, In Progress, Testing, and Done.

At the sprint review, the team demonstrates the completed features and collects stakeholder feedback. During the retrospective, the team identifies successful practices and improvements for the next sprint.

5.7 Definition of Done

A selected user story is considered complete when its implementation satisfies the acceptance criteria, required tests pass, critical defects are resolved, the code is reviewed, and the feature is ready for demonstration.

5.8 Conclusion

The sprint backlog converts high-priority user stories into practical development tasks. By selecting a manageable set of features and assigning clear responsibilities, the team can monitor progress, test completed work, and deliver a useful increment at the end of Sprint 1.

QUESTION 6: ESTIMATE EFFORT USING STORY POINTS

6.1 Introduction

Story-point estimation is an Agile technique used to estimate the relative effort required to complete a user story. Instead of estimating only the time required, the team considers complexity, uncertainty, implementation effort, and testing requirements.

Story points help the team compare different user stories and make informed sprint-planning decisions. They are not a direct measurement of hours or days. The actual time required depends on the team's skills, experience, capacity, and technical environment.

For the Cloud-Based Online Food Ordering and Delivery System, the Fibonacci sequence is used to estimate selected user stories.

6.2 Fibonacci Estimation Scale

The commonly used Fibonacci scale for story-point estimation is:

1, 2, 3, 5, 8, 13, 21

The increasing gaps between numbers reflect the uncertainty involved in estimating larger and more complex tasks.

- **1 point:** Very simple functionality with minimal complexity.
- **2 points:** Simple functionality requiring limited development and testing.
- **3 points:** Moderate functionality involving a few related tasks.
- **5 points:** More complex functionality involving several components or dependencies.
- **8 points:** High complexity or uncertainty requiring significant integration and testing.
- **13 points or more:** Very large or uncertain work that may need to be divided into smaller user stories.

6.3 Estimation Technique: Planning Poker

Planning Poker is a collaborative estimation technique used by Agile teams.

The process is as follows:

1. The product owner explains the user story and its acceptance criteria.
2. Team members discuss the required functionality and technical dependencies.
3. Each member independently selects a Fibonacci story-point estimate.
4. All estimates are revealed simultaneously.
5. If estimates differ, team members explain their reasoning.
6. The team discusses complexity, uncertainty, and testing needs.
7. The team agrees on a reasonable estimate.

This approach reduces individual bias and encourages shared understanding of the requirements.

6.4 Story-Point Estimates for Selected User Stories

User Story	Description	Story Points	Justification
US1	Customer registration	3	Requires form design, validation, database insertion, and testing.
US2	Customer login	3	Requires authentication, credential validation, and access control.
US4	View restaurants	5	Requires database retrieval, interface development, and error handling.
US5	View food menu	5	Requires menu data retrieval, price display, availability handling, and testing.
US7	Add items to cart	5	Requires cart management, quantity handling, and subtotal calculation.
US8	Modify shopping cart	3	Requires item removal, quantity updates, and recalculation.
US9	Place food order	5	Requires checkout validation, order creation, and database integration.
US10	Online payment	8	Requires payment gateway integration, transaction handling, and failure testing.
US11	Order confirmation	3	Requires order identifiers, status information, and confirmation display.
US12	Order tracking	5	Requires status updates, retrieval, and customer-facing display.

These estimates are illustrative and should be validated by the actual development team during Sprint Planning.

6.5 Justification for Sprint 1 Estimates

Sprint 1 includes customer registration, login, restaurant listing, and food menu display.

Customer registration is estimated at 3 story points because it requires a form, input validation, database operations, and basic error handling.

Customer login is estimated at 3 story points because the team must validate credentials, manage authentication, and test invalid login attempts.

Restaurant listing is estimated at 5 story points because it requires database design, restaurant data retrieval, interface development, and handling cases where no restaurants are available.

Food menu display is estimated at 5 story points because the system must retrieve menu items, display prices, indicate availability, and verify that the displayed information is correct.

The total estimate for Sprint 1 is:

3 + 3 + 5 + 5 = 16 story points.

This represents the relative estimated size of the selected work, not a guaranteed measure of productivity.

6.6 Sprint Capacity and Velocity

Sprint capacity represents the work a team can reasonably undertake during a sprint, considering availability, other responsibilities, and the complexity of the selected stories.

Velocity is the number of story points a team completes during a sprint. Since this is the team's first sprint, its actual velocity is not yet known. The proposed 16-point sprint should therefore be treated as a planning estimate rather than a proven team capacity.

After Sprint 1, the team can compare the completed story points with the original estimate and use the result to improve future sprint planning.

6.7 Benefits of Story-Point Estimation

Story-point estimation supports realistic planning, highlights complex features, improves team discussion, and helps identify stories that need further clarification. It also allows the team to learn from previous sprints and make better forecasts.

6.8 Conclusion

The Fibonacci sequence and Planning Poker provide a structured way to estimate the relative effort of user stories. By considering complexity, uncertainty, dependencies, and testing, the team can prepare a more realistic sprint backlog. The estimates should be reviewed and improved as the project progresses.

QUESTION 7: PRESENT THE SPRINT GOAL AND EXPECTED DELIVERABLES

7.1 Introduction

The sprint goal describes the main objective that the development team intends to achieve during a sprint. It gives the team a shared purpose and helps members coordinate their work. Expected deliverables are the working software features and related outputs that should be available at the end of the sprint.

For the Cloud-Based Online Food Ordering and Delivery System, the first sprint focuses on the initial customer-facing functionality. The sprint is planned for two weeks, and its scope includes registration, login, restaurant listing, and food menu display.

7.2 Sprint Goal

The sprint goal is:

To develop a working initial version of the Cloud-Based Online Food Ordering and Delivery System that enables customers to register, log in, view available restaurants, and browse food menus through a user-friendly interface.

This goal is specific because it identifies the features to be delivered. It is measurable because each feature can be tested against its acceptance criteria. It is achievable when the team has sufficient capacity and access to the required development environment. It is relevant because these features establish the foundation for the ordering process.

7.3 Selected User Stories

The following user stories are selected for Sprint 1:

US1 – Customer Registration: Customers can create accounts using valid personal details.

US2 – Customer Login: Registered customers can authenticate themselves and access the appropriate application functions.

US4 – View Restaurants: Customers can view the available restaurants and their relevant information.

US5 – View Food Menu: Customers can view food items, prices, and availability for a selected restaurant.

The selected stories are related because customers must be able to access the application and explore restaurants before proceeding to later features such as the shopping cart and checkout.

7.4 Expected Deliverables

1. Customer Registration Module

The registration module will provide a form for entering required customer information. It will validate input, prevent duplicate account creation where applicable, and display appropriate success or error messages.

2. Customer Login Module

The login module will allow registered customers to authenticate themselves. It will validate credentials, handle unsuccessful login attempts, and restrict access to authorized application functions.

3. Restaurant Listing Module

The restaurant listing module will retrieve and display available restaurant information. It will provide a clear interface and handle empty results or data retrieval errors.

4. Food Menu Display Module

The menu module will display food names, prices, and availability for the selected restaurant. The displayed information should correspond to the data stored in the system.

5. Database Integration

The database component will support customer accounts, restaurant records, and food menu information. Appropriate relationships and validation rules should be established to maintain data consistency.

6. Test Cases and Test Results

The testing deliverable will include test cases for valid registration, invalid input, successful login, failed login, restaurant listing, and menu display. The team will record the test results and correct defects identified during testing.

7. Sprint Demonstration

The team will demonstrate the working features during the sprint review. Stakeholders can evaluate the functionality and provide feedback that may be used to refine the product backlog.

7.5 Definition of Done

The sprint deliverables are considered complete when:

1. The selected features satisfy their agreed acceptance criteria.
2. Required functional tests have passed.
3. Critical defects in the delivered features have been resolved.
4. Database operations work correctly for the selected functionality.
5. The application can be demonstrated in the intended development environment.
6. Relevant code and documentation are reviewed.
7. Stakeholders can inspect the completed functionality during the sprint review.

7.6 Sprint Review

At the end of the sprint, the development team presents the working application to the product owner and stakeholders. The demonstration includes customer registration, login, restaurant browsing, and menu display.

Stakeholders can verify whether the features meet their expectations. Feedback may identify additional requirements, usability improvements, or corrections. Any new work is recorded in the product backlog and prioritized for a future sprint.

7.7 Sprint Retrospective

After the sprint review, the team conducts a retrospective to examine how the sprint was performed. Team members discuss what went well, what difficulties occurred, and what can be improved.

For example, the team may identify that clearer acceptance criteria reduced misunderstandings or that database integration required more time than expected. These observations can be used to improve estimation, communication, testing, and task allocation in the next sprint.

7.8 Expected Outcome

At the end of Sprint 1, the project should have a demonstrable foundation for customer access and restaurant browsing. The application will not yet include every feature required for the complete food ordering process. The shopping cart, order placement, payment processing, order confirmation, and delivery tracking can be developed in subsequent sprints.

This incremental approach allows stakeholders to review working software early and provide feedback before the project becomes more complex.

7.9 Conclusion

A clear sprint goal and realistic deliverables help the Agile team maintain focus and measure progress. By completing the initial customer-facing features, the team establishes a foundation for subsequent development. Sprint reviews and retrospectives support continuous feedback and improvement, helping the Cloud-Based Online Food Ordering and Delivery System evolve according to user needs.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25